01/07/26

Ordenação interna

→ Radix Sort
↳ baseado em raízes
↳ considera a posição dos dígitos

ordenação pela significância do dígito

completa a chave com 0 quando necessário

usa uma fila com a posição representando unidade, dezena, ...

parte do começo significativo para o mais significativo

faz lógica entre fila principal e estruturas auxiliares

Implementação do Radix Sort:

```
for (i = 0; i < tamanho; i++) {
    if (vetor[i] > maior)
        maior = vetor[i];
}

while (maior/exp > 0) {
    int bucket[10] = { 0 };
    for (i = 0; i < tamanho; i++)
        bucket[(vetor[i] / exp) % 10]++;
    for (i = 1; i < 10; i++)
        bucket[i] += bucket[i - 1];
    for (i = tamanho - 1; i >= 0; i--)
        b[--bucket[(vetor[i] / exp) % 10]] = vetor[i];
    for (i = 0; i < tamanho; i++)
        vetor[i] = b[i];
    exp *= 10;
}
```

→ encontra a raiz

Radix Sort

- Distribuição dos números 35 e 53 ➡ escritos na notação decimal como:
  - 35
    - 3 dezenas
    - 5 unidades
  - 53
    - 5 dezenas
    - 3 unidades
- Organize os elementos pela casa das dezenas ou pela casa das unidades
- Considerar Grupos (Dezenas, por exemplo)
  - Refinar processo
  - **Resultado:** Elementos organizados

Classificação,

Most significant digit (MSD)
least sig " " (LSD)

como se vai começar, pelo o mais significativo dígito primeiro

- Exemplificação: Dígito mais significativo
  - Considere uma lista de inteiros com o mesmo número de dígitos ➡ 24 23 22 37 12 92 86 33

Primeira etapa: P= a_i / base(10) **Classifica pelo dígito mais significativo.**

Fila	0	1	2	3	4	5	6	7	8	9
Listas encadeadas		12	24 23 22	37 33					86	92

← limitar listas em cada bucket

Resultado:12, 24, 23, 22, 37, 33, 86, 92

ordenado pelo MSD

respeita a ordem da entrada

Métodos de Ordenação: Interna

Exemplificação: Dígito mais significativo

Segunda etapa: P= a_i mod **base(10)** **Classifica grupo pelo dígito menos significativo**

Fila

0	1	2	3	4	5	6	7	8	9
	12	24 23 22	37 33					86	92

Fila: Estrutura auxiliar

0	1	2	3	4	5	6	7	8	9
		22	23	24					

Para lista com mais de um elemento

0	1	2	3	4	5	6	7	8	9
	12	22 23 24	33 37					86	92

radix sort no dígito menos significativo

o inverso também pode ser feito

Exemplificação: Dígito menos significativo

- Considere a lista de inteiros com o mesmo número de dígitos ➡ 25 57 48 37 12 92 86 33

Primeira etapa: P= a_i mod **base(10)** **Classifica pelo dígito menos significativo**

Fila

	0	1	2	3	4	5	6	7	8	9
Listas encadeadas			12 92	33		25	86	57 37	48	

Segunda etapa: P=(a_i/**base**) **Classifica pelo dígito mais significativo**

Para lista com mais de um elemento

Fila

0	1	2	3	4	5	6	7	8	9
	12	25	33 37	48	57			86	92

Arquivo classificado: 12, 25, 33, 37, 48, 57, 86, 92

Conclusão

caso 1: Dígitos distintos
Nunca serão periódicos
constante: sim

caso 2: Dígitos iguais
Sim, pois já estarão no buckets certo

- Análise
  - Tempo depende da quantidade de dígitos (m) e do número de elementos no arquivo (n)
    - Repetição: conjunto n
      - **for** (i= 0; i < n; i++)
      - percorrida *i* vezes (uma vez para cada elemento no arquivo)
    - Repetição: conjunto m
      - **for** (k = 0; k< m; k++)
      - percorrida *m* vezes (uma vez para cada dígito)

Tempo de Execução: $O(m.n) \rightarrow O(n), \forall m \in O(n)$.

O tempo depende tanto da quantidade como do número de dígitos no número

m não importa, tanto, uma vez que todos vão ter, que ser. Assim tratamos em O(1). Assim a complexidade é no geral O(n)

→ método estável
→ não há comparações
→ exige 2x mais memória que o quicksort
→ tempo é gasto para extrair dígitos

Counting sort

→ vetor de entrada de grande cada elemento é digitado como inteiro de 1 até K, onde K é maior valor
→ conta quantos elementos em uma posição i

Counting Sort: Exemplo

	Não Ordenado							
Inicial	2	5	3	0	2	3	0	7
Vetor A								
	Ordenado							
Auxiliar	0	0	0	0	0	0	0	0
Vetor B								
	Posições							
Contagem	0	0	0	0	0	0	0	0
Vetor C	0	1	2	3	4	5	6	7

Cada A[*i*] está entre *0* e *k* ➡ ***k*=7**

O(n)

Counting Sort: Exemplo

Não Ordenado

Inicial	2	5	3	0	2	3	0	7
Vetor A	*j*			Ordenado				
Auxiliar	0	0	0	0	0	0	0	0
Vetor B				Posições				
Contagem	0	0	1	0	0	0	0	0
Vetor C	0	1	2	3	4	5	6	7

uma barrinha

++

vamos entrando e vai somando 1 no array de contagem

Counting Sort: Exemplo

Não Ordenado

Inicial	2	5	3	0	2	3	0	7
Vetor A				Ordenado				*j*
Auxiliar	0	0	0	0	0	0	0	0
Vetor B				Posições				
Contagem	2	2	4	6	6	7	7	8
Vetor C	0	1	2	3	4	5	6	7

Determinar, para cada elemento *x*, o número de elementos menores ou iguais a *x*.

vai somando e acumulando com base na contagem

Counting Sort: Exemplo

Não Ordenado

Inicial	2	5	3	0	2	3	0	7
Vetor A						j		

Ordenado

Auxiliar	0	0	0	0	0	3	0	7
Vetor B								

Posições

Contagem	1	2	4	5	6	7	7	7
Vetor C	0	1	2	3	4	5	6	7

Determinar, para cada elemento x, o número de elementos menores ou iguais a x.

Vai varrer

↳ olhando da trás pra frente em j

Para cada valor na posição j da verifica a quantidade j do valor no auxiliar adicionada na posição somada no valor j e subtraindo

Counting Sort: Algoritmo e Análise

```
para i ← 0 até k faça
        C[i] ← 0                                  → 0
para j ← 1 até n faça
        C[A[j]] ← C[A[j]] + 1                     → 0
para i ← 1 até k faça
        C[i] ← C[i] + C[i − 1]                    → 0
para j ← n decrescendo até 1 faça
        B[C[A[j]]] ← A[j]                         → 0
        C[A[j]] ← C[A[j]] − 1                     → 0
```

comparação do laço não está dentro

- Quantas comparações? 0
  - Comparações para manter laços → não consideradas

Métodos de Ordenação: Interna

Counting Sort: Algoritmo e Análise

```
para i ← 0 até k faça                             → O(k)
        C[i] ← 0
para j ← 1 até n faça                             → O(n)
        C[A[j]] ← C[A[j]] + 1
para i ← 1 até k faça                             → O(k)
        C[i] ← C[i] + C[i − 1]
para j ← n decrescendo até 1 faça                 → O(n)
        B[C[A[j]]] ← A[j]
        C[A[j]] ← C[A[j]] − 1
```

k pode ser desconsiderado

- E o tempo de execução? $O(n+k)$ → $O(n)$

já que o que implica mais é n

Counting Sort: Algoritmo e Análise

```
para i ← 0 até k faça
        C[i] ← 0                                  → 0
para j ← 1 até n faça
        C[A[j]] ← C[A[j]] + 1                     → 0
para i ← 1 até k faça
        C[i] ← C[i] + C[i − 1]                    → 0
para j ← n decrescendo até 1 faça
        B[C[A[j]]] ← A[j]
        C[A[j]] ← C[A[j]] − 1                     → O(n)
```

→ tempo linear

Quantas trocas? $O(n)$

→ Estável
→ não faz comparações
→ usa muitos auxiliares
→ tempo i requerido para construir índice
→ não trabalha com outro tipo de dado
→ os auxiliares podem ter espaço não nulos

→ Se não ponto muitas comparações
→ algoritmo fica mais lento